

MESA ECG Sleep Stage Classification

Detailed Implementation & Architecture Report

Single-Channel ECG · 1D ResNet-34 · 5-Class Sleep Staging

Five Experimental Variants Compared

Version	File	Key Change
Version 3 (Best)	(3).ipynb	TRAIN oversample ×10, VAL/TEST no augmentation
Version 4 (Changed Aug Params)	(4).ipynb	TRAIN oversample ×6.4 (reduced factor)
Version 5 (No Oversample)	(5).ipynb	All splits use split() — no augmentation at all
Version 6 (Larger Batch)	(6).ipynb	BATCH_SIZE=32 (doubled from 16), same aug as V3
Version 7 (SMOTE)	(7).ipynb	Replaces manual augmentation with sklearn SMOTE

1. Project Overview

This project implements automatic sleep stage classification using a single ECG channel from the MESA (Multi-Ethnic Study of Atherosclerosis) polysomnography dataset. Five experimental notebook variants explore how different data balancing strategies and training configurations affect classification of 30-second ECG windows into five sleep stages.

1.1 Classification Target

Class ID	Sleep Stage	Description
0	Wake (W)	Subject is awake
1	N1 (NREM1)	Light sleep, transition from wake
2	N2 (NREM2)	Stable light sleep, sleep spindles
3	N3 (NREM3)	Deep slow-wave sleep
4	REM	Rapid Eye Movement sleep, dreaming

1.2 Shared Design Decisions (All Versions)

- **Single ECG channel only** — no EEG, EMG, or other sensors
- **30-second windows at 256 Hz** → 7,680 samples per window
- **1D ResNet-34 architecture** adapted for time-series ECG signals
- **Class-weighted Cross-Entropy loss** to handle natural class imbalance
- **Early stopping (patience=5)** to prevent overfitting
- **Stratified split BEFORE augmentation** to prevent data leakage

2. Dataset & Environment Setup

2.1 MESA Dataset

The MESA Sleep Study provides overnight polysomnography recordings in EDF format with corresponding XML annotation files. Each subject contributes multiple hours of continuous ECG recording annotated in 30-second epochs by expert scorers.

2.2 Configuration Parameters

Parameter	V3 / V5 / V7	V4	V6	Description
BATCH_SIZE	16	16	32	Mini-batch size for gradient updates
LR	1e-3	1e-3	1e-3	Initial learning rate
EPOCHS	30	30	30	Max training epochs
DROPOUT	0.3	0.3	0.3	Dropout rate in model head
patience	5	5	5	Early stopping patience
TRAIN_OVERSAMPLE_FACTOR	10	6.4	10	Augmentation target multiplier
TARGET_FS	256 Hz	256 Hz	256 Hz	ECG resampling target
EPOCH_SEC	30 s	30 s	30 s	Window duration

Parameter	V3 / V5 / V7	V4	V6	Description
SAMPLES_PER_EPOCH	7,680	7,680	7,680	Samples per window
NUM_WORKERS	0	0	0	DataLoader workers (safe for Windows)

2.3 Key Config Difference — Version 6

Version 6 increases BATCH_SIZE from 16 to 32. This requires more GPU VRAM but produces more stable gradient estimates per step, potentially reducing training noise. All other hyperparameters remain identical to Version 3.

2.4 New Parameters — Version 7 (SMOTE)

Parameter	Value	Description
SMOTE_K_NEIGHBORS	5	Number of nearest neighbors used by SMOTE to generate synthetic samples
SMOTE_OVERSAMPLE_FACTOR	10	target_per_class = min_class_count × 10 (same scale as V3)

3. Data Loading Pipeline (Cell 3) — Shared Across All Versions

All five versions share the identical data loading pipeline. The pipeline reads every EDF file, pairs it with its XML annotation, preprocesses the ECG signal, and organizes 30-second windows into per-class lists.

3.1 Processing Steps

Step 1 — File Discovery

The pipeline scans EDF_DIR for all .edf files. For each EDF, it constructs the expected XML path by appending '-nsrr.xml' to the patient ID stem. Files missing annotation pairs are skipped.

Step 2 — ECG Channel Extraction (pyEDFlib)

The code searches all channel labels for any name containing 'ECG', 'EKG', or 'CARDIAC'. The original sampling frequency is recorded for accurate resampling downstream.

Step 3 — Bandpass Filtering

A 4th-order zero-phase Butterworth bandpass filter removes baseline wander and high-frequency noise:

```
bandpass_filter(signal, lowcut=0.5 Hz, highcut=40.0 Hz, order=4)
```

Zero-phase `filtfilt()` is used to avoid phase distortion that would shift ECG peak timing.

Step 4 — Resampling to 256 Hz

`scipy.signal.resample()` brings all recordings to a uniform 256 Hz target. This ensures 7,680-sample windows regardless of the original acquisition hardware.

Step 5 — XML Annotation Parsing

The NSRR XML format stores sleep stage events with `EventType` containing 'stage'. The `EventConcept` field is parsed, mapped via `LABEL_MAP` to integers 0-4, and durations > 30s are handled by repeating labels.

Step 6 — Window Extraction & Z-Score Normalization

Each 30-second segment is sliced and per-window Z-score normalized (only when `std > 1e-6`):

```
window = (window - window.mean()) / window.std()
```

4. Stratified Data Splitting (Cell 4) — Shared Across All Versions

Splitting is always performed BEFORE augmentation. This prevents data leakage where synthetic samples derived from the same original window appear in both train and test sets.

Stage	Action	Result
Stage 1	<code>StratifiedShuffleSplit(test_size=0.10)</code>	Carve out 10% test set from 90% trainval
Stage 2	<code>StratifiedShuffleSplit(test_size=0.111)</code>	Split trainval into 80% train + 10% val

Stratification preserves per-class proportions identically across all three subsets, preventing accidental exclusion of rare classes from validation or test.

5. Augmentation & Balancing Strategy (Cell 5) — Main Differentiator

This is the primary experimental variable across the five versions. Versions 3, 4, 5, and 6 share the same manual augmentation primitives but differ in how aggressively they apply them. Version 7 replaces the entire approach with SMOTE.

5.1 Manual Augmentation Primitives (Versions 3, 4, 5, 6)

The `augment_ecg()` function applies the following stochastic transforms:

Transform	Parameters	Probability	Purpose
Gaussian Noise	<code>std ∈ [0.005, 0.03]</code>	Always	Simulates sensor noise and electrode artifacts

Transform	Parameters	Probability	Purpose
Amplitude Scaling	factor $\in$ [0.7, 1.3]	60% ($p > 0.4$)	Mimics inter-subject amplitude variability
Time Shift (Roll)	shift $\in$ $[-2 \times fs, +2 \times fs]$	60% ($p > 0.4$)	Breaks temporal alignment artifacts
Baseline Wander	freq $\in$ [0.05, 0.5] Hz, amp $\in$ [0.05, 0.2]	50% ($p > 0.5$)	Simulates respiration-induced drift
Re-normalization	Z-score per window	Always	Ensures synthetic samples remain scale-normalized

5.2 Balancing Logic — `augment_split()` vs `split()`

The `augment_split()` function computes a target count = $\text{ceil}(\text{min_class_count} \times \text{oversample_factor})$. Classes above target are randomly downsampled; classes below target receive synthetic samples from `augment_ecg()`. The `split()` function simply returns original samples without any modification.

5.3 Version-by-Version Comparison

Aspect	V3 (Best)	V4 (Aug Params)	V5 (No Oversample)	V6 (Larger Batch)	V7 (SMOTE)
Train function	<code>augment_split()</code>	<code>augment_split()</code>	<code>split()</code>	<code>augment_split()</code>	<code>smote_split()</code>
TRAIN oversample factor	10	6.4	N/A	10	10 (SMOTE_OVERSAMPLE_FACTOR)
Val/Test function	<code>split()</code>	<code>split()</code>	<code>split()</code>	<code>split()</code>	<code>split()</code>
Synthetic generation	Manual <code>augment_ecg()</code>	Manual <code>augment_ecg()</code>	None	Manual <code>augment_ecg()</code>	sklearn SMOTE (KNN interpolation)
Train class balance	Equal (all $\times 10$ min)	Equal (all $\times 6.4$ min)	Natural (imbalanced)	Equal (all $\times 10$ min)	Equal (all $\times 10$ min)
BATCH_SIZE	16	16	16	32	16
Expected minority recall	Highest	Moderate	Lowest	Similar to V3	High but different synthetic nature

5.4 Version 7 — SMOTE Deep Dive

SMOTE (Synthetic Minority Over-sampling Technique) is fundamentally different from the manual augmentation used in Versions 3-6. Instead of applying random signal transforms, SMOTE generates new samples by interpolating between existing samples in feature space.

SMOTE Algorithm

- **For each minority class sample x** , find its K nearest neighbors ($K=5$ by default)
- **Randomly select one neighbor x_{neighbor}**
- **Generate synthetic sample:** $x_{\text{synthetic}} = x + \text{rand}(0,1) \times (x_{\text{neighbor}} - x)$
- **Synthetic samples are tagged** with `PID='synthetic'` for traceability

Aspect	Manual Augmentation (V3/V4/V6)	SMOTE (V7)
Generation method	Random signal transforms (noise, scaling, shift, wander)	Linear interpolation between K nearest real samples in the feature space
Synthetic diversity	Controlled by transform parameters	Controlled by K neighbors and existing sample distribution
Physiological validity	Transforms are domain-inspired (noise, baseline wander)	Interpolated signals may not correspond to physiologically valid ECG
Implementation	Custom <code>augment_ecg()</code> function	<code>imblearn.over_sampling.SMOTE</code>
Downsampling	Random crop of majority classes	SMOTE only oversamples; majority classes remain at natural count or capped
Key parameters	<code>noise_std</code> , <code>amplitude factor</code> , <code>shift range</code>	<code>k_neighbors=5</code> , <code>sampling_strategy dict</code>

SMOTE Balancing Strategy in `smote_split()`

The strategy dict passed to SMOTE sets each class target to `max(target_count, current_count)`. This means SMOTE only adds samples — it never removes majority class samples. The resulting dataset may therefore be larger than in Versions 3/6 where majority classes are actively downsampled to match the target.

```
strategy = { c: max(target_count, counts[c]) for c in range(5) }
sm = SMOTE(sampling_strategy=strategy, k_neighbors=5, random_state=42)
X_res, Y_res = sm.fit_resample(X_raw, Y_raw)
```

6. PyTorch Dataset & DataLoaders (Cell 6)

All versions share the same `ECGDataset` class and `DataLoader` setup. The key transformation is adding a channel dimension required by 1D convolutions:

```
X shape: (N, 7680) → self.X shape: (N, 1, 7680) [batch, channel, time]
```

6.1 Version 6 — Effect of `BATCH_SIZE=32`

Aspect	BATCH_SIZE=16 (V3/V4/V5/V7)	BATCH_SIZE=32 (V6)
GPU VRAM usage	Lower (~2-3 GB for RTX 3050)	Higher (~3.5-4 GB, near VRAM limit)
Gradient estimate	Noisier (fewer samples per step)	Smoother (more samples per step)
Steps per epoch	More steps	Fewer steps (half as many)
Learning dynamics	More stochastic, can escape local minima	More stable, may converge more smoothly
Memory risk	Safe for 4 GB VRAM	May OOM on 4 GB VRAM with large datasets

7. Model Architecture — 1D ResNet-34 (Cell 7) — Shared

All five versions use the identical 1D ResNet-34 architecture. No architectural modifications were made between variants — only data preparation and training configuration differ.

7.1 Architecture Layer Table

Layer Group	Configuration	Output Shape (B, C, T)	Notes
Input	—	(B, 1, 7680)	Single ECG channel, 30s at 256 Hz
Stem	Conv1d(1→64, k=15, s=2) + BN + ReLU + MaxPool(s=2)	(B, 64, 1920)	k=15 captures slow ECG features
Layer 1	3 × BasicBlock(64→64, s=1)	(B, 64, 1920)	No downsampling, identity shortcut
Layer 2	4 × BasicBlock(64→128, s=2)	(B, 128, 960)	stride=2 halves time axis
Layer 3	6 × BasicBlock(128→256, s=2)	(B, 256, 480)	Deepest feature extraction stage
Layer 4	3 × BasicBlock(256→512, s=2)	(B, 512, 240)	Highest-level abstract features
GAP	AdaptiveAvgPool1d(1)	(B, 512, 1)	Collapses time → scalar per channel
Head	Flatten → Linear(512→256) + ReLU + Dropout(0.3) → Linear(256→5)	(B, 5)	Two-layer classifier

7.2 BasicBlock1D — Residual Unit

Each block applies two 3-point 1D convolutions with Batch Normalization and a skip connection:

```
out = ReLU(BN(Conv1d(x, k=3))) → Dropout(0.3) → BN(Conv1d(out, k=3))
output = ReLU(out + shortcut(x))
```

The shortcut uses a 1×1 Conv1d + BatchNorm when channel dimensions change; otherwise it is a plain identity mapping.

7.3 Model Statistics

Metric	Value
Architecture	1D ResNet-34
Total Parameters	~21 million
Input Shape	(batch, 1, 7680)
Output Shape	(batch, 5)
Activation	ReLU (inplace)
Normalization	BatchNorm1d after every Conv1d
Regularization	Dropout(0.3) in classification head
Weight Init	Kaiming Normal (He) for Conv1d and Linear layers

8. Training Loop (Cell 9) — Shared Across All Versions

8.1 Loss Function — Class-Weighted Cross-Entropy

Inverse-frequency class weights are computed from the training set distribution to penalize errors on rare classes more strongly:

```
class_weights[c] = total_samples / (5 × count_of_class_c)
```

This is particularly important for Version 5 (no oversampling) where the training set is naturally imbalanced.

8.2 Optimizer & Scheduler

Component	Setting	Rationale
Optimizer	AdamW (lr=1e-3, weight_decay=1e-4)	Decoupled L2 regularization for better generalization
LR Schedule	CosineAnnealingLR(T_max=30, eta_min=2e-5)	Smooth LR decay from 1e-3 to 2e-5 over all epochs
Gradient Clipping	clip_grad_norm_(max_norm=1.0)	Prevents gradient explosion in deep networks

Component	Setting	Rationale
Early Stopping	patience=5 epochs without val acc improvement	Stops training when model has plateaued

9. Evaluation (Cell 11) — Shared Across All Versions

The best checkpoint (highest validation accuracy) is loaded for test set inference. Metrics computed:

Metric	Tool / Formula	Purpose
Per-class Precision / Recall / F1	sklearn classification_report	Class-level diagnostic performance
Overall Accuracy	(correct) / total	Global performance
Cohen's Kappa (κ)	sklearn cohen_kappa_score	Agreement corrected for chance — standard PSG metric
Normalized Confusion Matrix	5×5 heatmap (row=true, col=predicted)	Error pattern analysis

10. Complete Five-Version Comparison

Aspect	V3 (Best)	V4	V5	V6	V7 (SMOTE)
Augmentation method	Manual (augment_ecg)	Manual (augment_ecg)	None	Manual (augment_ecg)	SMOTE (KNN interp)
Train oversample factor	10	6.4	N/A	10	10
BATCH_SIZE	16	16	16	32	16
Train class balance	Equal	Equal	Imbalanced (natural)	Equal	Equal (majority not reduced)
Synthetic samples	Yes (noise+scale+shift)	Yes (noise+scale+shift)	No	Yes (noise+scale+shift)	Yes (linear interpolation)
Key library	numpy/scipy	numpy/scipy	—	numpy/scipy	imblearn.SMOTE
Val/Test processing	No augmentation	No augmentation	No augmentation	No augmentation	No augmentation

Aspect	V3 (Best)	V4	V5	V6	V7 (SMOTE)
Primary research goal	Best overall model	Test reduced aug	Imbalance baseline	Test larger batch	Test feature-space synthesis

11. Full Pipeline Summary

Cell	Purpose	Shared?	Key Output
Cell 1	Environment verification	All	GPU/CPU status, import checks
Cell 2	Imports & CFG configuration	All (params vary)	CFG dict, LABEL_MAP, DEVICE
Cell 3	MESA data loading & preprocessing	All	class_samples[0..4], 7,680-sample windows
Cell 4	Stratified 80/10/10 split	All	split_samples{train/val/test}[0..4]
Cell 5	Augmentation & balancing	Differs per version	X_train, y_train, X_val, X_test
Cell 6	PyTorch Dataset + DataLoaders	All (batch differs)	train/val/test_loader
Cell 7	1D ResNet-34 definition	All	model (~21M parameters)
Cell 8	(Optional) Pretrained weights	All	Transfer learning init
Cell 9	Training loop	All	best_model.pt + history
Cell 10	Training curves	All	training_curves.png
Cell 11	Test evaluation	All	Report + confusion_matrix.png
Cell 12	Model persistence	All	Full checkpoint .pt file

12. Clinical & Research Context

Why Single-Channel ECG?

Traditional polysomnography (PSG) requires simultaneous EEG, EMG, EOG, airflow, and SpO2 — a complex clinic-based procedure. A single ECG electrode is feasible in consumer wearables and home monitoring. Heart rate variability (HRV) and R-R interval dynamics are well-established autonomic markers that differentiate sleep stages, making ECG a viable single-channel proxy.

Augmentation Strategy Impact

- **V3 (10× manual aug):** Best overall balance — strong minority class (N1, REM) recall due to aggressive oversampling. Test Acc: 83.15%, $\kappa=0.7544$
- **V4 (6.4× manual aug):** Reduced augmentation factor leads to weaker minority class recall (N1 recall drops to 23.4%). Test Acc: 80.41%, $\kappa=0.7131$
- **V5 (no aug):** Collapsed early (epoch 6) — natural class imbalance causes severe bias toward Wake/N2; N1 and REM essentially unlearned. Test Acc: 45.92%, $\kappa=0.2533$ (poor agreement)
- **V6 (batch=32):** Best overall performer — larger batch stabilises gradients, yielding the highest test accuracy and kappa across all versions. Test Acc: 84.25%, $\kappa=0.7671$
- **V7 (SMOTE):** Feature-space interpolation underperforms manual augmentation; physiologically invalid synthetic ECG segments likely hurt generalisation. Test Acc: 78.84%, $\kappa=0.6896$

The table below summarises the validation and test performance across all five experimental versions:

V3	Manual aug ×10, batch=16	83.93%	83.15%	0.7544	✓ Strong
V4	Manual aug ×6.4, batch=16	81.35%	80.41%	0.7131	△ Moderate
V5	No aug, natural imbalance	48.75%*	45.92%	0.2533	✗ Baseline
V6	Manual aug ×10, batch=32	83.70%	84.25%	0.7671	★ Best
V7	SMOTE ×10, batch=16	81.58%	78.84%	0.6896	▽ Weak

* V5 stopped at epoch 6 via early stopping; best val acc reflects a severely underfitted model biased toward majority classes.

Limitations

- **N1 is physiologically ambiguous** — ECG changes at Wake/N1/N2 boundaries are subtle
- **Single ECG cannot capture EEG-specific markers** (spindles, K-complexes, delta waves)
- **SMOTE-generated ECG segments** may not represent valid physiological waveforms
- **Performance may degrade** for subjects with arrhythmias underrepresented in MESA